

//Introduction to classes and objects

```
#include <iostream>
using namespace std;
class Rectangle {
public:
    int width;
    int height;
};
int main() {
    // Create Rectangle objects
    Rectangle rect1, rect2;
    rect1.width = 10;
    rect1.height = 15;
    rect2.width = 20;
    rect2.height = 40;
    cout << "Rect1's width = " << rect1.width << " and height = " << rect1.height << endl;
    cout << "Rect2's width = " << rect2.width << " and height = " << rect2.height << endl;
    return 0;
}
```

```
Rect1's width = 10 and height = 15
Rect2's width = 20 and height = 40
```

//class and objects 2

```
#include <iostream>
using namespace std;

class Rectangle {
public:
    int width;
    int height;
};
int main() {
    Rectangle rect;
    cout << "Rect's width = " << rect.width << " and height = " << rect.height << endl;

    return 0;
}
```

```
Rect's width = 32 and height = 1684955424
```

//Parameterized constructor with default values

```
#include <iostream>
```

```

using namespace std;
class Rectangle {
public:

    Rectangle(int a = 1, int b = 2) : width(a), height(b) {
        cout << "Constructor called with width = " << a << " and height = " << b << endl;
    }
    int area() {
        return width * height;
    }
private:
    int width;
    int height;
};
int main() {

    Rectangle rect1;

    Rectangle rect2(5);

    Rectangle rect3(3, 4);
    cout << "Area of rect1: " << rect1.area() << endl;
    cout << "Area of rect2: " << rect2.area() << endl;
    cout << "Area of rect3: " << rect3.area() << endl;
    return 0;
}

```

```

Constructor called with width = 1 and height = 2
Constructor called with width = 5 and height = 2
Constructor called with width = 3 and height = 4
Area of rect1: 2
Area of rect2: 10
Area of rect3: 12

```

//Copy constructor

```

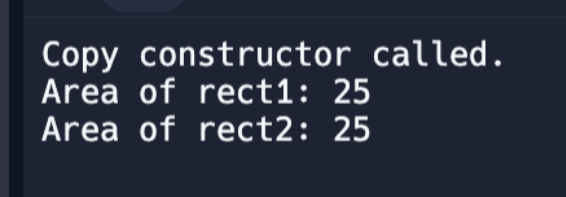
#include <iostream>
using namespace std;
class Rectangle {
public:
    Rectangle (int a, int b) : width(a), height(b) {}
    Rectangle(const Rectangle &obj) {
        width = obj.width;
        height = obj.height;
        cout << "Copy constructor called." << endl;
    }
}

```

```

int area() {
    return width * height;
}
private:
    int width;
    int height;
};
int main() {
    Rectangle rect1(5, 5);
    Rectangle rect2 = rect1; // Copy constructor is called here
    cout << "Area of rect1: " << rect1.area() << endl;
    cout << "Area of rect2: " << rect2.area() << endl;
    return 0;
}

```



```

Copy constructor called.
Area of rect1: 25
Area of rect2: 25

```

//Deconstructor

```

#include <iostream>
using namespace std;
class Rectangle {
public:
    Rectangle(int a, int b) : width(a), height(b) {}
    ~Rectangle() {
        cout << "Destructor called" << endl;
    }
    int area() {
        return width * height;
    }
private:
    int width;
    int height;
};
int main() {
    Rectangle rect1(5, 5);
    cout << "Area of rect1: " << rect1.area() << endl;
    return 0;
}

```

Area of rect1: 25
Destructor called

//Constructor overloading

```
#include <iostream>
using namespace std;

class Rectangle {
public:
    Rectangle () {
        width = 10;
        height = 10;
        cout << "Constructor #1 called" << endl;
        cout << "Rect's width = " << width << " and height = " << height << endl;
    }
    Rectangle (int a, int b) : width(a), height(b) {
        cout << "Constructor #2 called" << endl;
        cout << "Rect's width = " << width << " and height = " << height << endl;
    }
    int area() {
        return width * height;
    }
private:
    int width;
    int height;
};

int main() {
    Rectangle rect1;
    cout << "Area of rect1: " << rect1.area() << endl;
    Rectangle rect2(5, 5);
    cout << "Area of rect2: " << rect2.area() << endl;
    return 0;
}
```

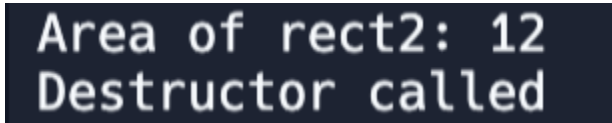
```
Constructor #1 called
Rect's width = 10 and height = 10
Area of rect1: 100
Constructor #2 called
Rect's width = 5 and height = 5
Area of rect2: 25
```

//Dynamic memory allocation

```
#include <iostream>
using namespace std;

class Rectangle {
public:
    Rectangle(int a, int b) : width(a), height(b) {}
    ~Rectangle() {
        cout << "Destructor called" << endl;
    }
    int area() {
        return width * height;
    }
private:
    int width;
    int height;
};

int main() {
    Rectangle* rect2 = new Rectangle(3, 4);
    cout << "Area of rect2: " << rect2->area() << endl;
    delete rect2;
    return 0;
}
```



```
Area of rect2: 12
Destructor called
```

//Accessing methods

```
#include <iostream>
using namespace std;

class Rectangle {
public:
    Rectangle(int a, int b) : width(a), height(b) {}
    ~Rectangle() {
        cout << "Destructor called" << endl;
    }
    int area() {
        return width * height;
    }
private:
    int width;
    int height;
};
```

```

int main() {
    Rectangle rect1(5, 5);
    cout << "Area of rect1: " << rect1.area() << endl;
    Rectangle* rect2 = new Rectangle(3, 4);
    cout << "Area of rect2 (using arrow operator): " << rect2->area() << endl;

    delete rect2;
    return 0;
}

```

```

Area of rect1: 25
Area of rect2 (using arrow operator): 12
Destructor called
Destructor called

```

//Full code

```

#include <iostream>
using namespace std;
class Rectangle {
public:
    Rectangle (int a, int b) : width(a), height(b) {}

    ~Rectangle() {
        cout << "Destructor called" << endl;
    }

    int area () {
        return (width * height);
    }
private:
    int width, height;
};

int main() {
    Rectangle rect1(5, 5);
    Rectangle* rect2 = new Rectangle(3, 4);
    cout << "Area of rect1: " << rect1.area() << endl;

    cout << "Area of rect2 (using dereference): " <<
    (*rect2).area() << endl;

    cout << "Area of rect2 (using arrow operator): " <<
    rect2->area() << endl;
}

```

```
    delete rect2;  
    return 0;  
}
```

```
Area of rect1: 25  
Area of rect2 (using dereference): 12  
Area of rect2 (using arrow operator): 12  
Destructor called  
Destructor called
```